CEE6501 — Lecture 11.2

Geometric Nonlinear Planar Truss Analysis

Learning Objectives

By the end of this lecture, you will be able to:

- explain the **corotational / Eulerian local coordinate system** for a geometrically nonlinear truss member
- compute the deformed member length and current direction cosines from the global nodal displacements
- compute the member axial force and transform it into the global member end-force vector $\mathbf{f}$
- explain why the global member relations are **geometrically nonlinear**
- interpret the **tangent stiffness matrix** and its main components
- explain the basic idea of the **Newton-Raphson method** for solving nonlinear equilibrium equations

Agenda

- Part 1 — Geometric Nonlinear Truss Setup
- Part 2 — Member Force-Displacement Relations in the Local System
- Part 3 — Local to Global Transformations
- Part 4 — In-Class Example (Given Displacements)
- Part 5 — Member Tangent Stiffness Matrix
- Part 6 — Newton-Raphson Method
- Part 7 — Example 1D Code

Part 1 — Geometric Nonlinear Truss Setup

What Stays the Same

Much of the overall structural setup remains the same as in the linear truss case.

We still need to:

- establish a **global coordinate system**
- number the **joints** and **members**
- identify the global displacement DOFs
- identify restrained and free coordinates

So the overall direct-stiffness workflow is familiar.

What changes is the way we define the **member local coordinate system** and the way we express the member force-displacement relations.

What Changes

In **linear truss analysis**, the member local coordinate system is fixed in the initial undeformed configuration.

In **geometrically nonlinear truss analysis**, it is more convenient to use a local coordinate system that:

- remains attached to the member
- translates with the member
- rotates with the member as the structure deforms

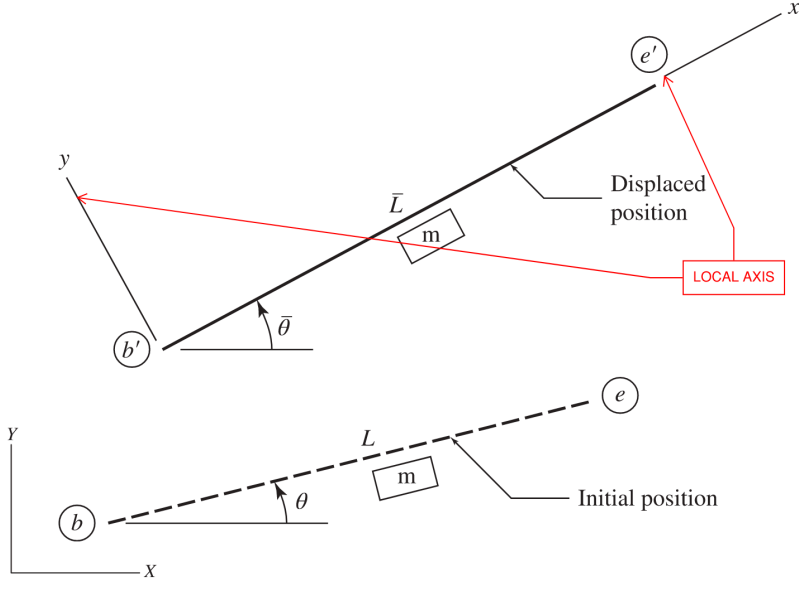
So the local x-axis is always directed along the member centroidal axis in its **current deformed state**.

This moving local system is often called a **corotational** or **Eulerian** local coordinate system.

Corotational / Eulerian Local Coordinate System

For a plane truss member:

- the origin of the local system is always at the **beginning node** of the member in its deformed state
- the local x-axis is directed along the **deformed member axis**
- the positive local y-axis is chosen so the coordinate system remains **right-handed**



Why This Coordinate Choice Helps

For a truss member, the only true member deformation is **axial stretching or shortening**.

By aligning the local axis with the **current deformed member direction**, we can write the member constitutive law in a very simple form.

So even though the structure moves and rotates in space, the member still behaves locally like an axial bar.

Part 2 — Member Force-Displacement Relations in the Local System

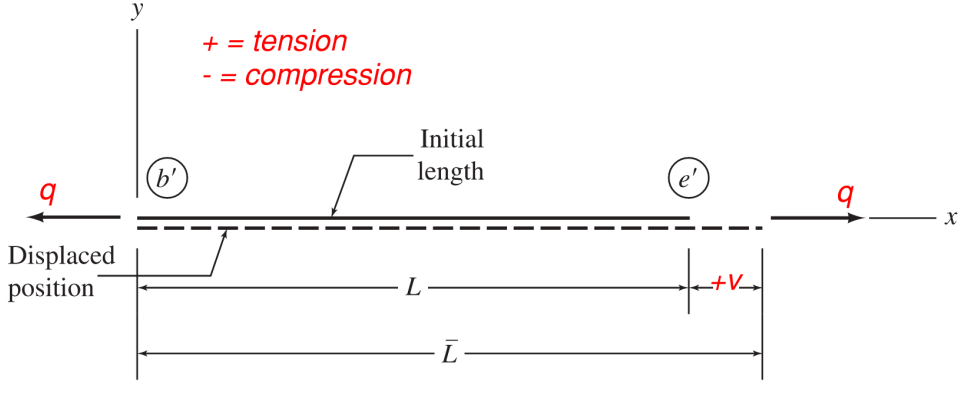
Axial Deformation in the Local System

Let a plane truss member have:

- undeformed length L
- deformed length $\bar{L}$

Then the member axial deformation is defined as

$$v = \bar{L} - L \tag{1}$$



With this sign convention:

- $v > 0$ corresponds to **elongation**
- $v < 0$ corresponds to **shortening**

Likewise, the member axial force q is taken as:

- positive in **tension**
- negative in **compression**

Note that Kassimali derivation notation in Chapter 10 is compression positive. This has been flipped in this lecture to better align with typical structural logic.

Stress, Strain, and Constitutive Law

For the truss member,

$$\sigma = \frac{q}{A} \qquad \text{and} \qquad \varepsilon = \frac{v}{L} \tag{2}$$

For a linearly elastic material,

$$\sigma = E\varepsilon \tag{3}$$

Substituting Eq. (2) into Eq. (3) gives the member force-displacement relation in the local system:

$$q = \left(\frac{EA}{L}\right) v \tag{4}$$

So locally, the member still behaves like an axial spring.

In [ ]:

```
import numpy as np

# Example material and member properties
E = 70e3       # MPa = N/mm^2
A = 645.2      # mm^2
L = 3000.0     # mm

# Local axial deformations
# Positive = elongation in tension-positive notation
u_vals = np.array([10.0, 0.0, -10.0])  # mm

# Axial force: positive = tension
q_vals = (E * A / L) * u_vals

for u, q in zip(u_vals, q_vals):
    state = 'tension' if q/1000 > 0 else ('compression' if q/1000 < 0 else 'ze
    print(f'u = {u:6.1f} mm  ->  q = {q/1000:10.1f} kN   ({state})')
```

Important Observation

Eq. (4) has the same form as the familiar linear axial bar relation.

The key point is that the **material law itself is still linear**.

The geometric nonlinearity enters elsewhere (Part 3):

- through the changing member length $\bar{L}$
- through the changing member orientation
- and through the transformation from local to global quantities

Note that because of the modified definition of the local coordinate system, only one degree of freedom (that is the member axial deformation) is now needed to completely specify the displaced position of the member.

Part 3 — Local to Global Transformations

Initial and Deformed Member Length

Suppose a member connects nodes b and e.

Using the **initial** nodal coordinates,

$$L = \sqrt{(X_e - X_b)^2 + (Y_e - Y_b)^2} \tag{5}$$

If the global nodal displacements are

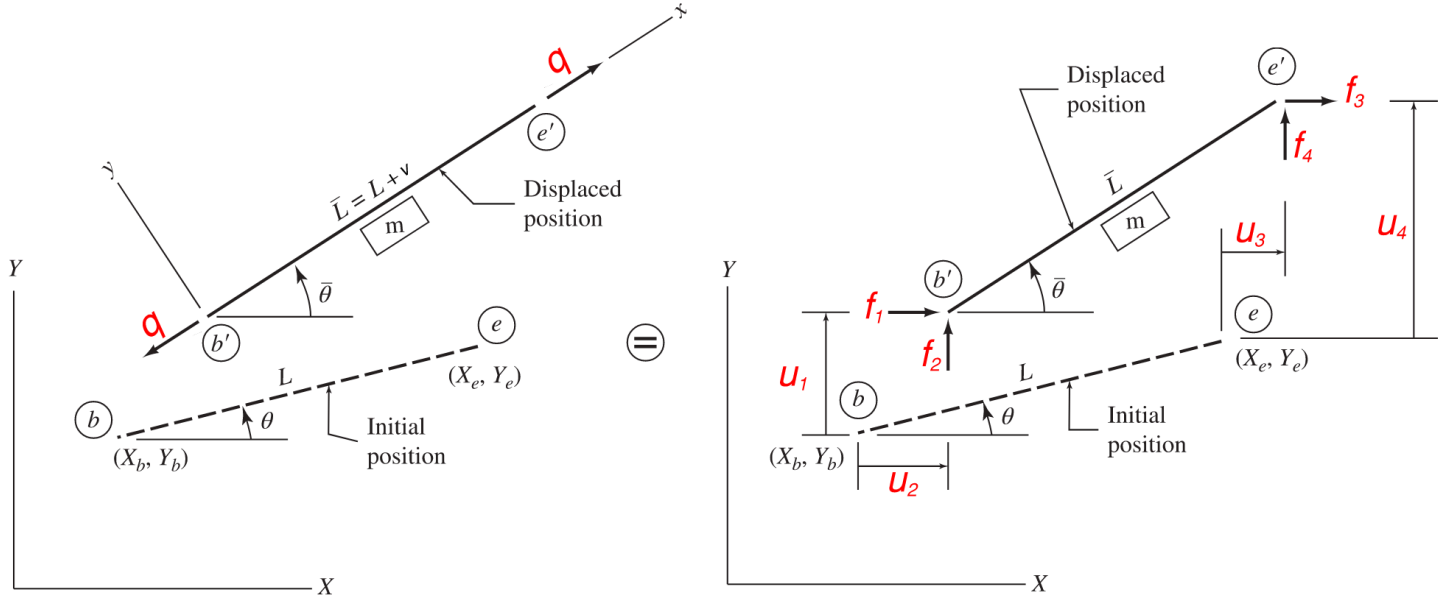
$$\mathbf{u} = [u_1 \; u_2 \; u_3 \; u_4]^T$$

with:

- u_1, u_2 = X- and Y-displacements (global) at node b
- u_3, u_4 = X- and Y-displacements (global) at node e

then the **deformed** member length is

$$\bar{L} = \sqrt{\left[(X_e + u_3) - (X_b + u_1)\right]^2 + \left[(Y_e + u_4) - (Y_b + u_2)\right]^2} \tag{6}$$



Current Direction Cosines

In the **deformed configuration**, the member orientation described by the current angle $\bar{\theta}$.

The corresponding direction cosines are

$$c_x = \cos\bar{\theta} = \frac{(X_e + u_3) - (X_b + u_1)}{\bar{L}} \tag{7}$$

$$c_y = \sin\bar{\theta} = \frac{(Y_e + u_4) - (Y_b + u_2)}{\bar{L}} \tag{8}$$

Here, $\bar{\theta}$ is measured **counterclockwise** from the positive global X-axis to the positive local x-axis of the member in its **deformed** position.

Unlike in linear analysis, these quantities are **not constant**. Because they depend on the current nodal displacements, they change continuously as the structure deforms.

Note: In 2D analysis, these rotation terms are often written more compactly as $c_x = c$ and $c_y = s$ (see Trusses in Lecture 4).

In [ ]:

```
# Example: compute deformed length and direction cosines from nodal displaceme

Xb, Yb = 0.0, 0.0
Xe, Ye = 4000.0, 3000.0

u1, u2 = 20.0, 10.0
u3, u4 = 80.0, -40.0

L0 = np.sqrt((Xe - Xb)**2 + (Ye - Yb)**2)
Lbar = np.sqrt(((Xe + u3) - (Xb + u1))**2 + ((Ye + u4) - (Yb + u2))**2)
cx = ((Xe + u3) - (Xb + u1)) / Lbar
cy = ((Ye + u4) - (Yb + u2)) / Lbar
u_local = L0 - Lbar

print(f'Initial length L      = {L0:.3f} mm')
print(f'Deformed length Lbar  = {Lbar:.3f} mm')
print(f'c_x = {cx:.6f}')
print(f'c_y = {cy:.6f}')
print(f'Local axial deformation u = {u_local:.3f} mm')
```

From Local Axial Force to Global End Forces

At node b, the global end-force components are the components of the local axial force q along the global axes:

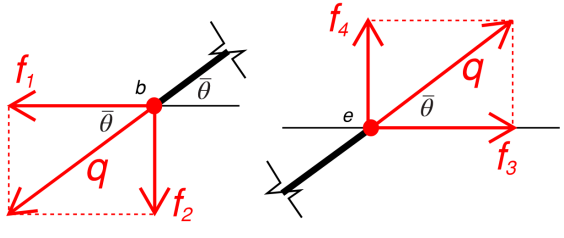
$$f_1 = -c_x q \tag{9a}$$

$$f_2 = -c_y q \tag{9b}$$

At node e, the equal-and-opposite components are

$$f_3 = c_x q \tag{9c}$$

$$f_4 = c_y q \tag{9d}$$



Transformation Form

So the global member end-force vector is

$$\mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} = \begin{bmatrix} -c_x \\ -c_y \\ c_x \\ c_y \end{bmatrix} q \tag{10}$$

We can write Eq. (10) more compactly as

$$\mathbf{f} = \mathbf{T}^T q \tag{11}$$

where the 1×4 transformation matrix is

$$\mathbf{T} = \begin{bmatrix} -c_x & -c_y & c_x & c_y \end{bmatrix} \tag{12}$$

In [ ]:

```
# Continue from the previous example

q = (E * A / L0) * u_local
T = np.array([[-cx, -cy, cx, cy]])
f = T.T * q

print(f'Local axial force q = {q/1000:.3f} kN')
print('\nTransformation matrix T =')
print(T)

print('\nGlobal end-force vector f =')
for i, val in enumerate(f.flatten(), start=1):
    print(f'f_{i} = {val/1000:.1f} kN')
```

Conceptual Summary of the Member Relations

To evaluate the member global end-force vector $\mathbf{f}$ from known global nodal displacements $\mathbf{u}$, the steps are:

1. compute the initial length L using Eq. (5)
2. compute the deformed length $\bar{L}$ using Eq. (6)
3. compute the current direction cosines c_x, c_y using Eqs. (7) and (8)
4. compute the local axial deformation $v = \bar{L} - L$ using Eq. (1)
5. compute the local axial force q from Eq. (4)
6. transform q to the global end-force vector $\mathbf{f}$ using Eq. (11)

Why the Global Member Relations Are Nonlinear

Even though the local constitutive law

$$q = \left(\frac{EA}{L}\right) v \tag{4}$$

is linear, the **global** member relations are geometrically nonlinear because:

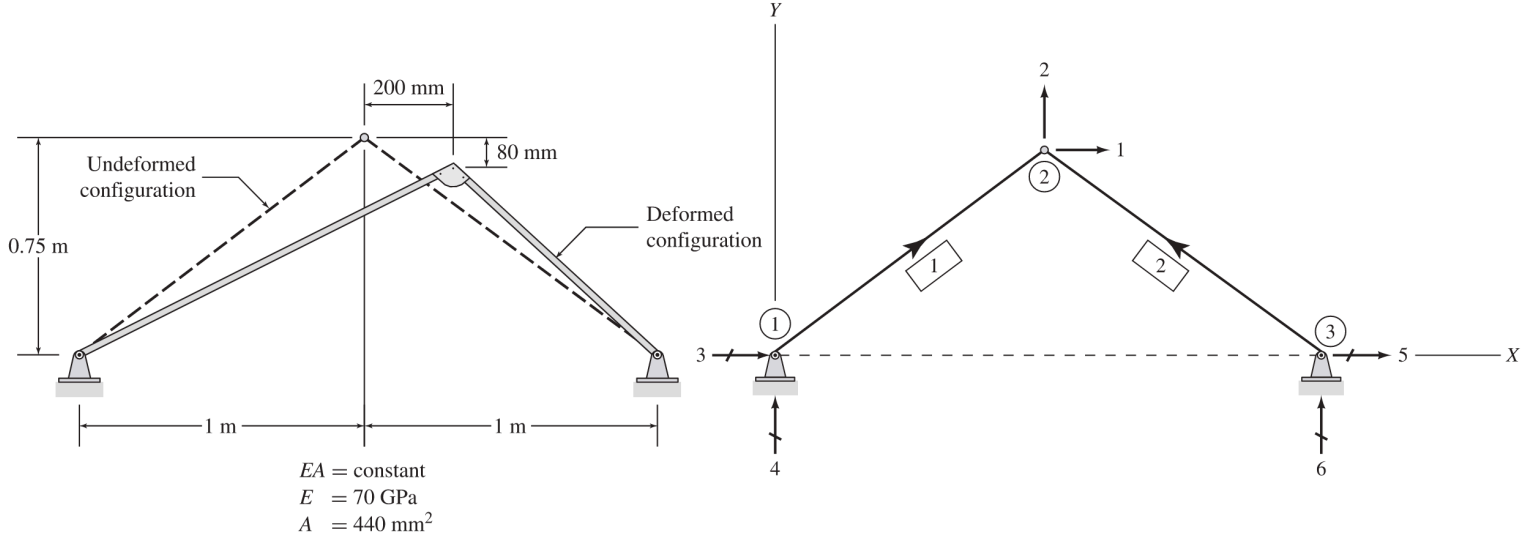
- $v = \bar{L} - L$ depends nonlinearly on the global nodal displacements
- c_x and c_y depend on the current deformed orientation
- the transformation matrix $\mathbf{T}$ therefore changes during deformation

So the member force vector $\mathbf{f}$ is ultimately a nonlinear function of the global displacement vector $\mathbf{u}$.

Part 4 — In-Class Example (Given Displacements)

Example Problem

By using geometrically nonlinear analysis, determine the joint loads P that cause the two-member truss to deform into the configuration shown



In [ ]:

```python
import numpy as np

def truss_deformed_geometry(start_point, end_point, deformations):
    Xb, Yb = start_point
    Xe, Ye = end_point
    u1, u2, u3, u4 = deformations

    L0 = np.sqrt((Xe - Xb)**2 + (Ye - Yb)**2)
    Lbar = np.sqrt(((Xe + u3) - (Xb + u1))**2 + ((Ye + u4) - (Yb + u2))**2)
    cx = ((Xe + u3) - (Xb + u1)) / Lbar
    cy = ((Ye + u4) - (Yb + u2)) / Lbar
    u_local = Lbar - L0

    return {
        "L0": L0,
        "Lbar": Lbar,
        "cx": cx,
        "cy": cy,
        "u_local": u_local,
    }
```

In [ ]:

```
# Calculate member geometric parameters
m1 = truss_deformed_geometry((0, 0), (1000, 750), (0, 0, 200, -80))
m2 = truss_deformed_geometry((0, 0), (-1000, 750), (0, 0, 200, -80))

E = 70e3  # MPa = N/mm^2
A = 440.0   # mm^2

# calculate local axial forces
q1 = (E * A / m1["L0"]) * m1["u_local"]
q2 = (E * A / m1["L0"]) * m2["u_local"]

print(f'Member 1: q = {q1/1000:.3f} kN')
print(f'Member 2: q = {q2/1000:.3f} kN')
```

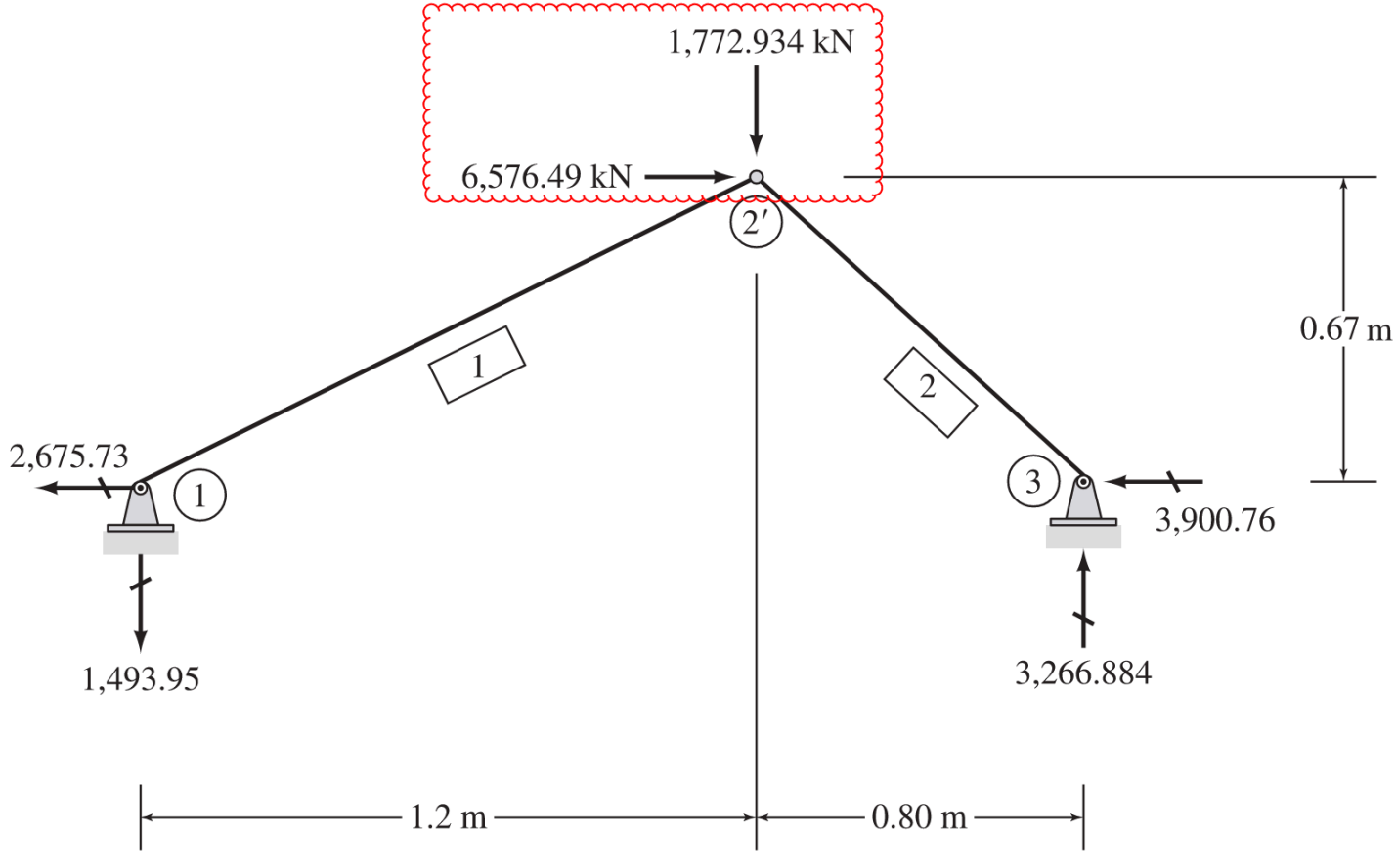
In [ ]:

```
# Tranform to global forces
T1 = np.array([[-m1["cx"], -m1["cy"], m1["cx"], m1["cy"]]])
T2 = np.array([[-m2["cx"], -m2["cy"], m2["cx"], m2["cy"]]])
f1 = T1.T * q1
f2 = T2.T * q2

print("Global end forces (kN)")
print(
    f"Member 1: "
    f"f1 = {f1[0,0]/1000:.1f}   "
    f"f2 = {f1[1,0]/1000:.1f}   "
    f"f3 = {f1[2,0]/1000:.1f}   "
    f"f4 = {f1[3,0]/1000:.1f}"
)

print(
    f"Member 2: "
    f"f1 = {f2[0,0]/1000:.1f}   "
    f"f2 = {f2[1,0]/1000:.1f}   "
    f"f3 = {f2[2,0]/1000:.1f}   "
    f"f4 = {f2[3,0]/1000:.1f}"
)
```

Add together f3 and f4 for both members to get X and Y forces at node 2



Part 5 — Member Tangent Stiffness Matrix

Why We Need a Tangent Stiffness Matrix

In the previous part, we expressed the member global end-force vector $\mathbf{f}$ as a nonlinear function of the global displacement vector $\mathbf{u}$.

That is useful when the displaced configuration is already known.

But in most analysis problems, the external loading is known, and the unknown is the displaced configuration itself. This requires solving a system of **simultaneous nonlinear equations**.

These problems are typically solved **iteratively** by repeatedly linearizing the force-displacement relation about the current configuration.

That linearized relation introduces the **member tangent stiffness matrix**.

Incremental Force-Displacement Relation

For small increments about the current deformed state, the member force-displacement relation can be written in differential form as

$$\Delta \mathbf{f} = \mathbf{K}_t \, \Delta \mathbf{u} \tag{12}$$

where:

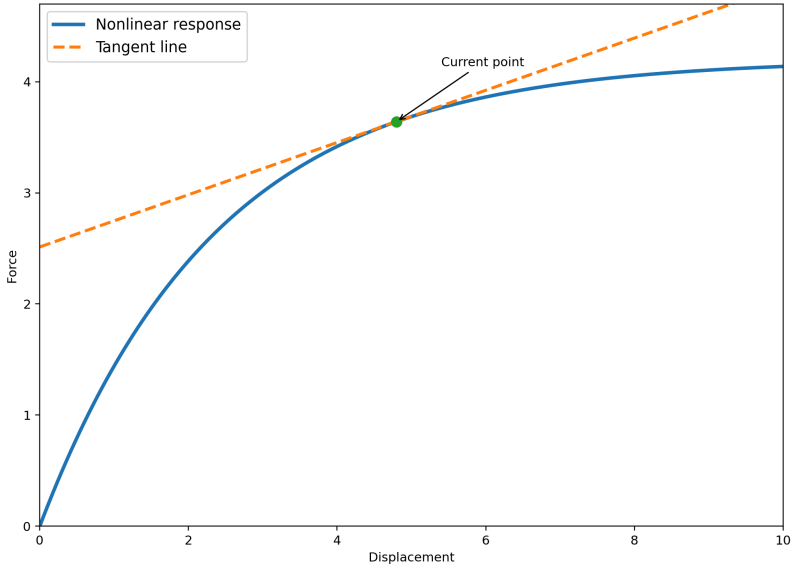
- $\Delta \mathbf{f}$ is the increment in the member global end-force vector
- $\Delta \mathbf{u}$ is the increment in the member global displacement vector
- $\mathbf{K}_t$ is the **member tangent stiffness matrix** in the global coordinate system

So $\mathbf{K}_t$ gives a **local linear approximation** of the nonlinear member response at the current configuration. In other words, it tells us how a small change in displacement produces a small change in force at that point.

Linearizing a Nonlinear Response

Tangent stiffness matrix, $\mathbf{K}_t$, gives a **local linear approximation** of the nonlinear response.

- over a small increment, we approximate the nonlinear curve as linear
- the slope of that local approximation is the **tangent stiffness**
- For large increments the error in the linear approximation is clear



Definition of the Tangent Stiffness Matrix

By definition,

$$\mathbf{K}_t = \left[\frac{\partial f_i}{\partial u_j}\right] \qquad i, j = 1, 2, 3, 4 \tag{14}$$

So each entry of $\mathbf{K}_t$ tells us how one component of the member end-force vector changes with respect to one component of the member displacement vector (i.e., the Jacobian).

$$\mathbf{K}_t = \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \frac{\partial f_1}{\partial u_2} & \frac{\partial f_1}{\partial u_3} & \frac{\partial f_1}{\partial u_4} \\ \frac{\partial f_2}{\partial u_1} & \frac{\partial f_2}{\partial u_2} & \frac{\partial f_2}{\partial u_3} & \frac{\partial f_2}{\partial u_4} \\ \frac{\partial f_3}{\partial u_1} & \frac{\partial f_3}{\partial u_2} & \frac{\partial f_3}{\partial u_3} & \frac{\partial f_3}{\partial u_4} \\ \frac{\partial f_4}{\partial u_1} & \frac{\partial f_4}{\partial u_2} & \frac{\partial f_4}{\partial u_3} & \frac{\partial f_4}{\partial u_4} \end{bmatrix} \tag{15}$$

Where the Terms Come From

Recall that the member global end-force vector can be written as

$$\mathbf{f} = \mathbf{T}^T q \tag{11}$$

where both:

- the local axial force in the member, q
- and the transformation matrix, $\mathbf{T}$

depend on the current member geometry.

So when we differentiate $\mathbf{f}$ with respect to $\mathbf{u}$, two kinds of terms appear:

- terms from how the **axial force magnitude** changes
- terms from how the **member orientation** changes

These lead naturally to the two parts of the tangent stiffness matrix.

Example of One Partial Derivative

$$\mathbf{K}_t = \begin{bmatrix} \boxed{\dfrac{\partial f_1}{\partial u_1}} & \dfrac{\partial f_1}{\partial u_2} & \dfrac{\partial f_1}{\partial u_3} & \dfrac{\partial f_1}{\partial u_4} \\ \dfrac{\partial f_2}{\partial u_1} & \dfrac{\partial f_2}{\partial u_2} & \dfrac{\partial f_2}{\partial u_3} & \dfrac{\partial f_2}{\partial u_4} \\ \dfrac{\partial f_3}{\partial u_1} & \dfrac{\partial f_3}{\partial u_2} & \dfrac{\partial f_3}{\partial u_3} & \dfrac{\partial f_3}{\partial u_4} \\ \dfrac{\partial f_4}{\partial u_1} & \dfrac{\partial f_4}{\partial u_2} & \dfrac{\partial f_4}{\partial u_3} & \dfrac{\partial f_4}{\partial u_4} \end{bmatrix} \tag{15}$$

For example, since $f_1 = -c_x q$, Eq. (9a), differentiating with respect to u_1 gives

$$\frac{\partial f_1}{\partial u_1} = -c_x \left(\frac{\partial q}{\partial u_1} \right) - q \left(\frac{\partial c_x}{\partial u_1} \right) \tag{16}$$

Deriving $\dfrac{\partial q}{\partial u_1}$

Step 1: Write the governing equations

From Eq. (4), the local constitutive law is

$$q = \left(\frac{EA}{L}\right) v \tag{4}$$

From Eq. (1), the axial deformation is

$$v = \overline{L} - L \tag{1}$$

From Eq. (6), the deformed length is

$$\overline{L} = \sqrt{\left[(X_e + u_3) - (X_b + u_1)\right]^2 + \left[(Y_e + u_4) - (Y_b + u_2)\right]^2} \tag{6}$$

From Eq. (7), the direction cosine is

$$c_x = \cos\bar{\theta} = \frac{(X_e + u_3) - (X_b + u_1)}{\bar{L}} \tag{7}$$

Step 2: Apply the chain rule to q

To find $\dfrac{\partial q}{\partial u_1}$, we apply the chain rule using Eqs. (4), (1), (6), and (7).

From Eq. (4),

$$\frac{\partial q}{\partial u_1} = \left(\frac{EA}{L}\right)\frac{\partial v}{\partial u_1}$$

From Eq. (1),

$$\frac{\partial v}{\partial u_1} = \frac{\partial \overline{L}}{\partial u_1}$$

because L is constant.

Therefore,

$$\frac{\partial q}{\partial u_1} = \left(\frac{EA}{L}\right)\frac{\partial \overline{L}}{\partial u_1} \tag{17}$$

Step 3: Differentiate the deformed length

Only the first squared term depends on u_1, so

$$\frac{\partial \bar{L}}{\partial u_1} = \frac{1}{2}\Big(\big[(X_e + u_3) - (X_b + u_1)\big]^2 + \big[(Y_e + u_4) - (Y_b + u_2)\big]^2\Big)^{-1/2} \cdot 2\big[(X_e + u_3) - (X_b + u_1)\big](-1)$$

which simplifies to

$$\frac{\partial \bar{L}}{\partial u_1} = -\frac{(X_e + u_3) - (X_b + u_1)}{\bar{L}}$$

Step 4: Substitute the direction cosine expression

Using Eq. (7), we obtain

$$\frac{\partial \bar{L}}{\partial u_1} = -c_x$$

Step 5: Final

Substituting back into Eq. (17), the expression for $\dfrac{\partial q}{\partial u_1}$, gives

$$\frac{\partial q}{\partial u_1} = \frac{EA}{L}(-c_x)$$

Therefore,

$$\boxed{\frac{\partial q}{\partial u_1} = -\frac{EA}{L}c_x} \tag{18}$$

Deriving $\dfrac{\partial c_x}{\partial u_1}$

Step 1: Write the governing equations

From Eq. (6), the deformed length is

$$\overline{L} = \sqrt{\left[(X_e + u_3) - (X_b + u_1)\right]^2 + \left[(Y_e + u_4) - (Y_b + u_2)\right]^2} \tag{6}$$

From Eqs. (7) and (8), the direction cosines are

$$c_x = \cos\bar{\theta} = \frac{(X_e + u_3) - (X_b + u_1)}{\bar{L}} \tag{7}$$

$$c_y = \sin\bar{\theta} = \frac{(Y_e + u_4) - (Y_b + u_2)}{\bar{L}} \tag{8}$$

Step 2: Apply the quotient rule to c_x

To find $\dfrac{\partial c_x}{\partial u_1}$, we differentiate Eq. (7) with respect to u_1.

Let

$$N = (X_e + u_3) - (X_b + u_1)$$

so that

$$c_x = \frac{N}{\bar{L}}$$

Using the quotient rule

$$\frac{\partial c_x}{\partial u_1} = \frac{\bar{L}\dfrac{\partial N}{\partial u_1} - N\dfrac{\partial \bar{L}}{\partial u_1}}{\bar{L}^2}$$

Since

$$\frac{\partial N}{\partial u_1} = -1$$

this becomes

$$\frac{\partial c_x}{\partial u_1} = \frac{-\bar{L} - N\dfrac{\partial \bar{L}}{\partial u_1}}{\bar{L}^2} \tag{19}$$

Step 3: Differentiate the deformed length

From Eq. (6),

$$\frac{\partial \bar{L}}{\partial u_1} = -\frac{(X_e + u_3) - (X_b + u_1)}{\bar{L}} = -\frac{N}{\bar{L}}$$

Substituting this into Eq. (19) gives

$$\frac{\partial c_x}{\partial u_1} = \frac{-\bar{L} - N\left(-\frac{N}{\bar{L}}\right)}{\bar{L}^2}$$

which simplifies to

$$\frac{\partial c_x}{\partial u_1} = \frac{-\bar{L} + \frac{N^2}{\bar{L}}}{\bar{L}^2} = \frac{-\bar{L}^2 + N^2}{\bar{L}^3}$$

Step 4: Rearrange and substitute

From Eq. (6),

$$\bar{L}^2 = N^2 + \left[(Y_e + u_4) - (Y_b + u_2)\right]^2$$

Rearranging gives

$$-\bar{L}^2 + N^2 = -\left[(Y_e + u_4) - (Y_b + u_2)\right]^2$$

Substituting in the expression for $\frac{\partial c_x}{\partial u_1}$

$$\frac{\partial c_x}{\partial u_1} = -\frac{\left[(Y_e + u_4) - (Y_b + u_2)\right]^2}{\bar{L}^3} \tag{20}$$

Step 5: Final

Substituting the expression for c_y, Eq. (8), into Eq. 20.

We obtain:

$$\boxed{\frac{\partial c_x}{\partial u_1} = -\frac{c_y^2}{\bar{L}}} \tag{21}$$

Final Expression for the First Term of the Tangent Stiffness

Substituting Eqs. (18) and (21), into Eq. (16),

$$\frac{\partial f_1}{\partial u_1} = -c_x \left(\frac{\partial q}{\partial u_1} \right) - q \left(\frac{\partial c_x}{\partial u_1} \right) \tag{16}$$

gives

$$\frac{\partial f_1}{\partial u_1} = -c_x \left(-\frac{EA}{L} c_x \right) - q \left(-\frac{c_y^2}{\bar{L}} \right)$$

which simplifies to

$$\boxed{\frac{\partial f_1}{\partial u_1} = \left(\frac{EA}{L} \right) c_x^2 + \left(\frac{q}{\bar{L}} \right) c_y^2}$$

Explicit Form of the Member Tangent Stiffness Matrix

Carrying out all of the partial derivatives gives

$$\mathbf{K}_t = \frac{EA}{L}\begin{bmatrix} c_x^2 & c_xc_y & -c_x^2 & -c_xc_y \\ c_xc_y & c_y^2 & -c_xc_y & -c_y^2 \\ -c_x^2 & -c_xc_y & c_x^2 & c_xc_y \\ -c_xc_y & -c_y^2 & c_xc_y & c_y^2 \end{bmatrix} + \frac{q}{\bar{L}}\begin{bmatrix} c_y^2 & -c_xc_y & -c_y^2 & c_xc_y \\ -c_xc_y & c_x^2 & c_xc_y & -c_x^2 \\ -c_y^2 & c_xc_y & c_y^2 & -c_xc_y \\ c_xc_y & -c_x^2 & -c_xc_y & c_x^2 \end{bmatrix} \tag{22}$$

Compact Form (Kassimali)

It is often convenient to write Eq. (22) in the compact form

$$\mathbf{K}_t = \left(\frac{EA}{L}\right)\mathbf{T}^T\mathbf{T} + q\,\mathbf{G} \tag{23}$$

where the **geometric matrix** $\mathbf{G}$ is

$$\mathbf{G} = \frac{1}{\bar{L}}\begin{bmatrix} c_y^2 & -c_xc_y & -c_y^2 & c_xc_y \\ -c_xc_y & c_x^2 & c_xc_y & -c_x^2 \\ -c_y^2 & c_xc_y & c_y^2 & -c_xc_y \\ c_xc_y & -c_x^2 & -c_xc_y & c_x^2 \end{bmatrix}$$

This form separates the tangent stiffness into a **material part** and a **geometric part**.

Physical Meaning of the Two Terms

In Eq. (23):

- $\left(\frac{EA}{L}\right)\mathbf{T}^T\mathbf{T}$ is the **material stiffness** contribution
- $q\,\mathbf{G}$ is the **geometric stiffness** contribution

The first term comes from the constitutive law of the bar.

The second term comes from the fact that the member direction changes as the structure deforms.

So even if the material remains linearly elastic, the tangent stiffness still depends on the current force state and current geometry.

Check Against the Linear Limit

In the initial undeformed configuration:

- $\bar{L} = L$
- c_x and c_y reduce to the original direction cosines
- the axial force $q = 0$

So Eq. (22) reduces to

$$\mathbf{K}_t = \left(\frac{EA}{L}\right)\mathbf{T}^T\mathbf{T} + \mathbf{0}$$

which is exactly the conventional linear stiffness matrix for a 2D truss member.

That is a useful consistency check.

Truss Element Derivation from Lecture 3

Calculating the Global Stiffness Matrix, $\boldsymbol{K}$

The global element stiffness matrix is obtained by **rotating the local axial stiffness** into the global coordinate system:

$$\boldsymbol{K} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & \cos\theta & -\sin\theta \\ 0 & 0 & \sin\theta & \cos\theta \end{bmatrix} \cdot \frac{EA}{L} \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \cos\theta & \sin\theta & 0 & 0 \\ -\sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & \cos\theta & \sin\theta \\ 0 & 0 & -\sin\theta & \cos\theta \end{bmatrix}$$

Carrying out the matrix multiplication yields the **closed-form global stiffness matrix**:

$$\boldsymbol{K} = \frac{EA}{L} \begin{bmatrix} \cos^2\theta & \cos\theta\sin\theta & -\cos^2\theta & -\cos\theta\sin\theta \\ \cos\theta\sin\theta & \sin^2\theta & -\cos\theta\sin\theta & -\sin^2\theta \\ -\cos^2\theta & -\cos\theta\sin\theta & \cos^2\theta & \cos\theta\sin\theta \\ -\cos\theta\sin\theta & -\sin^2\theta & \cos\theta\sin\theta & \sin^2\theta \end{bmatrix}$$

$$\mathbf{K}_t = \left(\frac{EA}{L}\right)\mathbf{T}^T\mathbf{T} + \mathbf{0} = \frac{EA}{L}\begin{bmatrix} c_x^2 & c_xc_y & -c_x^2 & -c_xc_y \\ c_xc_y & c_y^2 & -c_xc_y & -c_y^2 \\ -c_x^2 & -c_xc_y & c_x^2 & c_xc_y \\ -c_xc_y & -c_y^2 & c_xc_y & c_y^2 \end{bmatrix} + \mathbf{0}$$

Part 6 - Newton-Raphson Method

Main Computational Idea: Convergence to Equilibrium

In linear analysis, we typically solve once:

$$\mathbf{K}\mathbf{u} = \mathbf{f}$$

In geometrically nonlinear analysis, the stiffness depends on the current configuration, so the problem is instead posed as finding a configuration that satisfies equilibrium:

$$\mathbf{r}(\mathbf{u}) = \mathbf{0}$$

where $\mathbf{r}(\mathbf{u})$ is the **residual** or **out-of-balance force vector**.

So the solution process becomes iterative:

1. guess a configuration
2. evaluate the internal resisting force
3. compute the residual
4. correct the configuration
5. repeat until the residual is sufficiently small

This iterative point of view is the foundation of the Newton-Raphson method.

Equilibrium at an Iteration

At iteration i, at the free DOFs, we compare:

- the applied external load vector $\mathbf{f}_f$ (this does not change)
- the current internal resisting force vector $\mathbf{f}_{f,\text{int}}^{(i)}$

The residual is defined as:

$$\mathbf{r}^{(i)} = \mathbf{f}_f - \mathbf{f}_{f,\text{int}}^{(i)} \tag{1}$$

Our goal is to drive $\mathbf{r}^{(i)} \to \mathbf{0}$. Equilibrium is satisfied when:

$$\mathbf{r}^{(i)} = \mathbf{0}$$

In other words, when the external and internal forces balance.

Local Linearization Idea

Because the exact equilibrium equations are nonlinear, we do not solve them in one step.

Instead, at the current configuration for the free DOFs, $\mathbf{u}_f^{(i)}$, we linearize the problem and solve for a correction increment:

$$\mathbf{r}^{(i)} = \mathbf{K}_{ff,t}^{(i)} \, \Delta\mathbf{u}_f^{(i)} \tag{2}$$

where:

- $\mathbf{K}_{ff,t}^{(i)}$ is the **tangent stiffness matrix** evaluated at the current configuration
- $\Delta\mathbf{u}_f^{(i)}$ is the **incremental correction** for that iteration
- $\mathbf{r}^{(i)}$ is the residual being driven toward zero

This is the nonlinear analogue of solving a linear stiffness equation.

First Approximation

A common way to begin is to use the stiffness of the undeformed configuration to obtain the first approximate configuration.

Using the initial tangent stiffness,

$$\mathbf{f}_f^{(1)} = \mathbf{K}_{ff,t}^{(0)} \, \mathbf{u}_f^{(1)}$$

in which $\mathbf{K}_{ff,t}^{(0)}$ denotes the structure tangent stiffness matrix evaluated in the undeformed configuration (i.e., $\mathbf{u}_f = \mathbf{u}_f^{(0)} = \mathbf{0}$).

Solving for $\mathbf{u}_f^{(1)}$ gives a linearized first estimate of the displacement state:

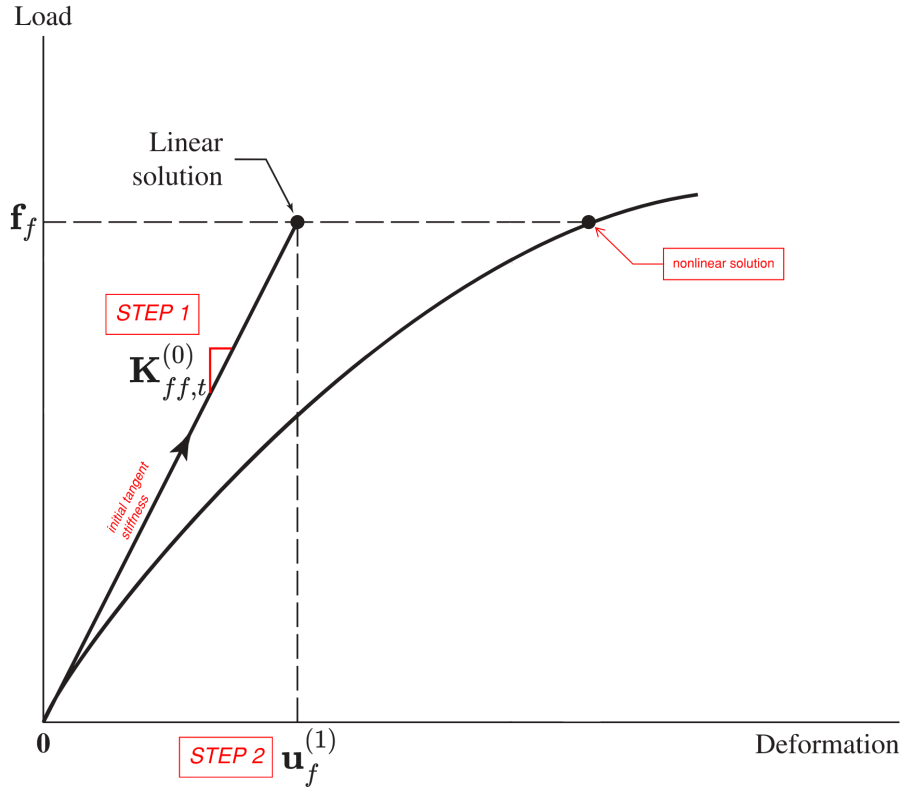
$$\mathbf{u}_f^{(1)} = \mathbf{K}_{ff,t}^{-1(0)} \mathbf{f}_f^{(1)}$$

However, $\mathbf{u}_f^{(1)}$ is only an approximate configuration, because the joint equilibrium equations are not necessarily satisfied at this stage. In general, the internal resisting force associated with $\mathbf{u}_f^{(1)}$ will not exactly match the applied load.

Only in the linear case do we obtain exact equilibrium in a single step. This is the type of problem considered earlier in the course (Lecture 3 and 4), where the stiffness remains constant and no geometric stiffness term is included.

First Approximation - Graphical Interpretation

1. Initialize with the structure tangent stiffness matrix at 0 deformation, $\mathbf{K}_{ff,t}^{(0)}$
2. Solve for $\mathbf{u}_f^{(1)}$ as a linearized first estimate of the displacement state



Residual and Correction Step

To correct the approximate solution at the free DOFs, $\mathbf{u}_f^{(i)}$, we evaluate the structure's internal joint force vector at the free DOFs corresponding to this configuration:

$$\mathbf{f}_{f,\text{int}}^{(i)} = \mathbf{f}_{f,\text{int}}(\mathbf{u}_f^{(i)})$$

We then subtract it from the applied joint load vector at the free DOFs, $\mathbf{f}_f$, to calculate the unbalanced joint force vector:

$$\mathbf{r}^{(i)} = \mathbf{f}_f - \mathbf{f}_{f,\text{int}}^{(i)} \tag{1}$$

This residual is treated as the load that still needs to be balanced.

The correction vector, $\Delta\mathbf{u}_f^{(i)}$, is then obtained by solving with the new tangent stiffness at the current configuration, where $\mathbf{K}_{ff,t}^{(i)}$ now represents the structure tangent stiffness matrix for the free DOFs evaluated at $\mathbf{u}_f^{(i)}$:

$$\mathbf{r}^{(i)} = \mathbf{K}_{ff,t}^{(i)}\,\Delta\mathbf{u}_f^{(i)} \tag{2}$$

$$\Delta\mathbf{u}_f^{(i)} = \mathbf{K}_{ff,t}^{-1(i)}\mathbf{r}^{(i)}$$

Update of the Configuration

After solving for the correction increment, we update the displacement state:

$$\mathbf{u}_f^{(i+1)} = \mathbf{u}_f^{(i)} + \Delta\mathbf{u}_f^{(i)} \tag{3}$$

Iteration Procedure

3. evaluate internal force $\mathbf{f}_{f,\text{int}}^{(i)}$ at configuration $\mathbf{u}_f^{(i)}$
4. compute residual $\mathbf{r}^{(i)}$

$$\boxed{\mathbf{r}^{(i)} = \mathbf{f}_f - \mathbf{f}_{f,\text{int}}^{(i)}} \tag{1}$$

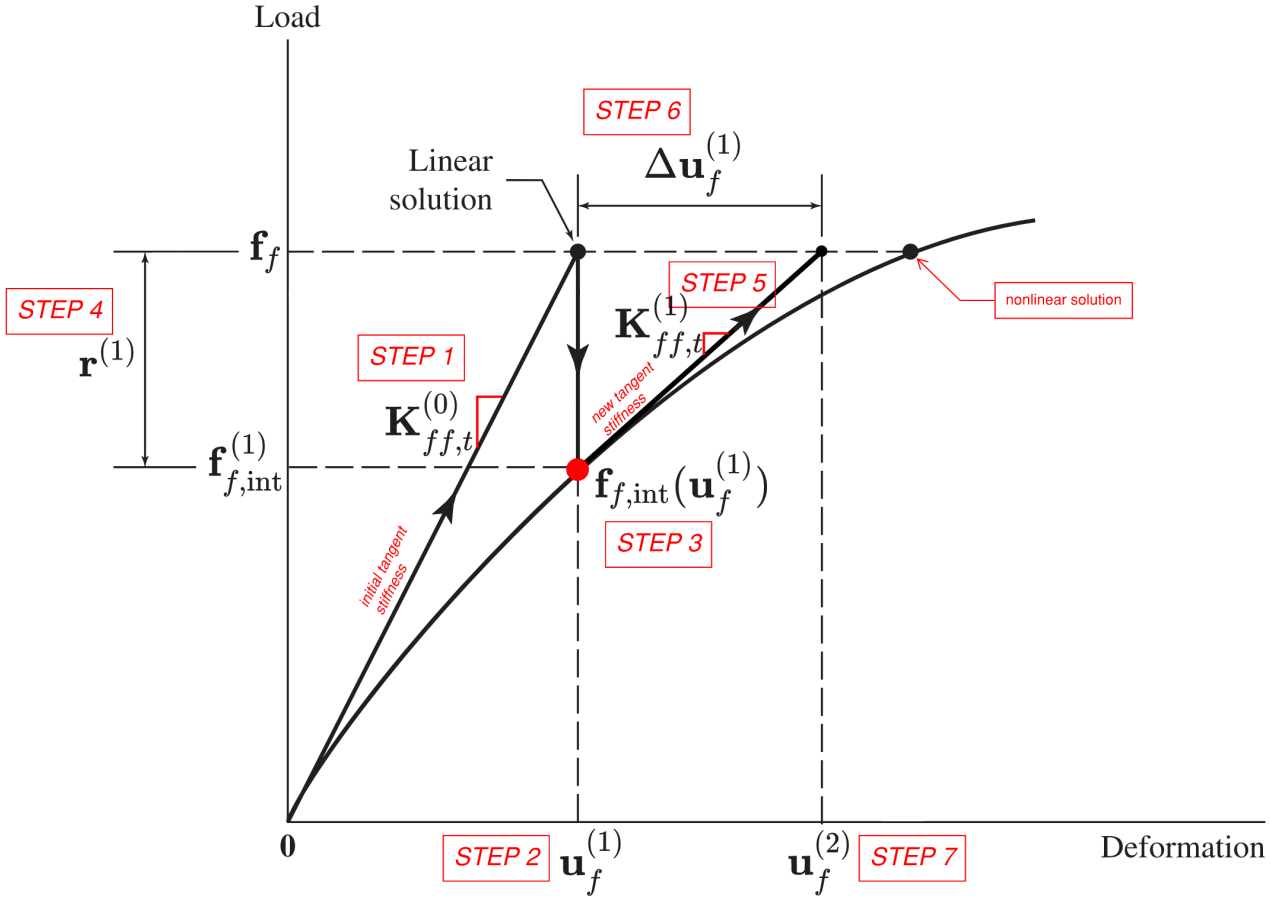
5. calculate new tangent stiffness matrix $\mathbf{K}_{ff,t}^{(i)}$ at configuration $\mathbf{u}_f^{(i)}$
6. solve for correction $\Delta\mathbf{u}_f^{(i)}$

$$\boxed{\Delta\mathbf{u}_f^{(i)} = \mathbf{K}_{ff,t}^{-1(i)}\mathbf{r}^{(i)}} \tag{2}$$

7. update to obtain $\mathbf{u}_f^{(i+1)}$ for the next iteration

$$\boxed{\mathbf{u}_f^{(i+1)} = \mathbf{u}_f^{(i)} + \Delta\mathbf{u}_f^{(i)}} \tag{3}$$

One Iteration - Graphical Interpretation



Convergence Criterion

We need a numerical test to decide when the iterations have converged.

One common displacement-based criterion, at iteration i, is:

$$\sqrt{\frac{\sum_{j=1}^{NDOF}\left(\Delta u_{f,j}^{(i)}\right)^2}{\sum_{j=1}^{NDOF}\left(u_{f,j}^{(i)}\right)^2}} \leq e$$

where e is a prescribed tolerance, e.g., 0.01 for a 1% change.

So convergence is declared when the correction becomes small relative to the current displacement magnitude.

Algorithm Summary

Newton-Raphson for geometrically nonlinear analysis can be written as:

1. initialize with tangent stiffness matrix at 0 deformation, $\mathbf{K}_{ff,t}^{(0)}$
2. solve for $\mathbf{u}_f^{(1)}$ as a linearized first estimate of the displacement state
3. compute internal force $\mathbf{f}_{f,\text{int}}^{(i)}$
4. compute residual $\mathbf{r}^{(i)} = \mathbf{f}_f - \mathbf{f}_{f,\text{int}}^{(i)}$
5. form the new tangent stiffness $\mathbf{K}_{ff,t}^{(i)}$
6. solve $\Delta\mathbf{u}_f^{(i)} = \mathbf{K}_{ff,t}^{-1(i)}\mathbf{r}^{(i)}$
7. update $\mathbf{u}_f^{(i+1)} = \mathbf{u}_f^{(i)} + \Delta\mathbf{u}_f^{(i)}$
8. check convergence
9. if not converged, repeat steps 3-8

At each iteration, we solve a local linear problem that moves us closer to nonlinear equilibrium.

Part 7 - Example 1D Code

Open `NewtonRaphson_1D.ipynb` in the `../../Code/L11` folder

Wrap-Up

In this lecture, we:

- introduced the **corotational / Eulerian local coordinate system** for a plane truss member
- derived the nonlinear member force relations in terms of the global nodal displacements
- introduced the **member tangent stiffness matrix**
- introduced the **Newton-Raphson method** for solving the nonlinear equilibrium equations

Next: we will solve a simple geometric nonlinear truss problem.